

1 What simulation is

Mimicking a real system over time; the next best thing to observing reality. Use when **queueing** + **shared resources** + **randomness** interact $\Rightarrow$ no analytical solution. **Pros**: models messy reality; allows uncertainty / non-stationary behaviour; animation aids verification & debugging; convincing to stakeholders. **Cons**: only **estimates**, not exact answers (each run = 1 random sample) $\Rightarrow$ needs statistical design & analysis. *This is why input modelling, V&V and output analysis exist.*

Simulation gives exact answers
Each run is one random sample
Captures stochastic + complex interactions

F
T
T

2 System classification (3 binary splits)

Stochastic (follows prob. distributions, unpredictable) vs **Deterministic** (output fixed given inputs).
 $\rightarrow$ **Dynamic** (state changes over time) vs **Static** (snapshot; e.g. Monte Carlo).
 $\rightarrow$ **Continuous** (smooth, ∞ states) vs **Discrete-event** (state changes only at event times).

Course systems = **Stochastic** + **Dynamic** + **Discrete** = DES. (Bank, airport, production line.)

DES state changes only at events
Monte Carlo is a static model
Continuous system has ∞ states

T
T
T

3 DES vocabulary (Rossetti)

- **Entity** – object of interest, flows through (customer, aircraft).
- **Resource** – operating object, constrains flow; *requested* $\rightarrow$ *seized* $\rightarrow$ *held* $\rightarrow$ *released* (teller, gate).
- **Process** – entity needs a resource to do an activity.
- **Event** – instant state change. **State var** – system now. **Queue** – waiting entities. **Attribute** – per-entity (entryTime). **Clock** – jumps event $\rightarrow$ event.

AnyLogic: Source $\rightarrow$ Queue $\rightarrow$ Service (=Seize-Delay-Release) $\rightarrow$ ResourcePool, SelectOutput, Sink.

4 Event clock / time advance (Ros 4.4)

DES uses **next-event time advance**: the clock *jumps* straight to the next event time, skipping the “dead” intervals where nothing happens (bank opens t=0, first arrival t=2 $\Rightarrow$ skip 0-2). **Events**: arrival, begin service, departure (service completion). The run is documented as an **event log** (time, event, comment). If all arrival & service times were known up front the system could be solved by hand – but they’re random, generated as the sim runs.

Two defs (Rossetti): (1) “the imitation of the operation of a real-world process or system over time”; (2) “a numerical technique for conducting experiments on a digital computer...describing the behaviour of a system over time.”

5 DEGREE problem-solving process

- Define the problem – solve the *right* problem.
- Establish performance measures – coherent w/ problem.
- Generate alternative solutions.
- Rank alternative solutions.
- Evaluate & iterate – until model fidelity ok.
- Execute & evaluate the solution – check real benefits.

First “D” = Phase 1 of the methodology.

6 General Simulation Methodology – 6 phases

Ph.1 Problem definition + objectives/planning (SMART KPIs) + conceptual model (context diagram, flow chart, activity diagram).
Ph.2 Data collection & model building (parallel) + coding; gates **Verified?** then **Validated?** – loop back if No.
Ph.3 Experimental design + production runs & analysis.
Ph.4 Decision **More runs?** – loop to Ph.3 if Yes.
Ph.5 Document programs & report results.
Ph.6 Implementation.

Know all 6; course = Ph.1–5. **Loop-backs**: verify/validate (in Ph.2) and “more runs?” (Ph.4).

7 Verification / Validation / Calibration

Verification = “built the model *right*?” $\rightarrow$ model vs **specification** (no bugs).
Validation = “built the *right* model?” $\rightarrow$ model vs **reality**.
Calibration = tune parameters so output matches reality.
Structural assumptions (system represented right; check w/ observation+experts) vs **Data assumptions** (data reliable; combined datasets need **homogeneity** test; dist. estimation needs i.i.d. sample).

Verification = model vs reality
Validation = model vs reality
Dist. estimation needs i.i.d. data

F
T
T

8 Conceptual diagrams

Context diagram / **flow chart**: boundary, inputs from environment, outputs, broad flow.
Activity diagram: entity flow – enter queue, **seize** resource (start activity), duration (e.g. expo(3) min), **release** (end), dispose. Each element = one AnyLogic block.

9 Bottlenecks & performance mgmt

Bottleneck = a point of congestion where workloads arrive faster than the process can handle; the **slowest stage**, caps system throughput (chain = slowest member); correct target for investment. Upgrading a non-bottleneck does nothing. *Can* be eliminated, but only by adding capacity – may be too costly.
Measures: **throughput** (units/time; slowest op governs), **operation time** (value-adding time at a resource), **queue/waiting time**, **lead/turnaround time** (start $\rightarrow$ finish; airport = turnaround), **utilisation** (operation time / available time).

10 S.M.A.R.T. KPIs (Doran 1981)

Specific (precisely defined) · Measurable (metric: quality/quantity/cost/time) · Attainable (actionable, achievable) · Relevant (appropriate to role, realistic) · Time-bound (agreed time scale).

SMART: “turnaround <75 min, per operating day.” **Not SMART**: “make airport more efficient.”

11 Input modelling (Ros 6.1–6.4)

Two input types: *Controlled/known* (you set: #gates, #staff) vs *Uncertain* (need distribution: arrival/service times).
8-step process: 1. document process; 2. plan collection; 3. collect data; 4. graphical+statistical analysis; 5. hypothesise dists; 6. estimate params; 7. check goodness of fit; 8. check output sensitivity.
Empirical vs theoretical: theoretical preferred if fits (smooth, full range); empirical always usable but *limited to observed range* – use carefully.
Ranges: ∞ both = Normal; positive = Exp, Gamma, Erlang, Weibull, Lognormal; bounded = Uniform, Beta, Triangular.

Cautions: don’t fit continuous to discrete data; no data $\Rightarrow$ expert opinion (Uniform needs only min/max); test sensitivity before collecting more.

12 Goodness of fit

χ^2 (**Pearson**) & Kolmogorov–Smirnov: both give test stat + **p-value**. p = chance of result $\geq$ as extreme under H_0 . If p < **significance level** (1% / 5%) $\Rightarrow$ reject fit.

TRAP: tests can *reject* a bad fit but **cannot rank** competing fits. χ^2 #bins: too few = over-aggregated; too many = inflated error.

Data Distribution Fitter: tests 9 dists, picks best by **minimising MSE** (separate from hypothesis tests).

χ^2 /KS can rank which fit is best
Low p-value $\Rightarrow$ reject the fit
Fit continuous dist to discrete data

F
T
F

13 Randomness (Ros 2.1–2.3)

Variates from pseudo-random streams; each run = 1 realisation. **Seed/stream** $\Rightarrow$ reproducibility. **Common Random Numbers (CRN)** across 2 configs $\Rightarrow$ fairer paired comparison. **Rossetti**: “random input $\Rightarrow$ random output” – output = true performance + unknown noise.

14 Random-variate generation – inverse transform

Generate a value from any dist F_X using a uniform $U \sim [0, 1]$:

$$\text{draw } U, \quad X = F_X^{-1}(U)$$

Works because if $X = F^{-1}(U)$ then $U = F(X)$ is uniform on $[0, 1]$. Needs F strictly increasing (invertible); works for *continuous* and *discrete* dists. $F(x) = \text{Prob}\{X \leq x\}$ (the CDF).

15 AnyLogic – Process Modelling blocks

- **Source** – agents enter; holds the arrival scheme; map arrivals to agent params.
- **Sink** – agents leave; model fails if they can’t exit; can have several.
- **Delay** – holds agent for a set time (fixed or stochastic dist) = an operation’s duration.
- **Queue** – buffers agents if next block busy; has *capacity*; FIFO default; .size() = # queueing; “exit on timeout” $\rightarrow$ outTimeout port.
- **Seize / Release** – bracket operations needing a resource; seize first, release after; must be paired.
- **Service** = Seize + Delay + Release combined. **ResourcePool** = the resource units. **SelectOutput** = routing/branching.

Stats gotchas: tick “Force statistics collection” on Service; Queue<Agent> has *no* timeHere() – store **entryTime** on the agent, use time()**–entryTime**; TimeMeasureEnd mean via timeTotal.distribution.mean(); ResourcePool block name must match Java reference exactly.

16 AnyLogic – arrival rate convention

Arrival rate $\lambda \Rightarrow$ **exponential interarrival times**. “Rate = 10/hour” $\equiv$ interarrival time $\sim$ exponential(10) with mean $1/\lambda = 0.1$ h. Set arrivals by Rate or Interarrival time – same thing. Service “rate μ ” $\Rightarrow$ service time $\sim$ exp, mean $1/\mu$.

TRAP: a stated arrival *rate* is NOT a fixed gap – it’s exponential (random) interarrivals. Rate 10/h $\neq$ one arrival every 6 min exactly.

17 Queueing basics (M/M/1)

Single server, Poisson arrivals λ , exp service μ , FIFO.

$$\rho = \frac{\lambda}{\mu} \text{ (utilisation),} \quad \text{steady state needs } \rho < 1$$

$$L_q = \frac{\rho\lambda}{\mu-\lambda} \text{ (avg \# in queue),} \quad L_s = L_q + \rho \text{ (in system)}$$

Worked (ATM): $\lambda=10/\text{h}$, $\mu=20/\text{h}$. $\rho = \frac{10}{20} = 0.5$. $L_q = \frac{0.5 \cdot 10}{20 - 10} = 0.5$ in queue. $\rho < 1 \Rightarrow$ stable.

18 What is a model? (abstraction)

A model = a **representation of a system**, considering only **relevant aspects** (abstraction). By definition a **simplification** – but detailed enough to draw *valid conclusions*. Different models of the same system are needed to study different changes. Solve the problem safely in the “risk-free world of models”, then map back to reality.

19 Three simulation methods (abstraction level)

- **System Dynamics (SD)** – *high* abstraction (macro/strategic): aggregates, stocks & flows, feedback loops.
- **Discrete-Event (DES)** – *medium* / *medium-low* (tactical/operational): process flowchart, entities + resources. **This course**.
- **Agent-Based (ABM)** – *any* level: define individual agents’ behaviour (often state-charts); global behaviour *emerges*.

Multi-method models combine these. Choosing the right abstraction level is critical; usually start high, add detail as needed.

14 Output analysis – why replications

1 run is noisy (Job Shop KPI: 129–151 min, ±8% around 140). ⇒ run many reps (*same params, diff seeds*), report **across-replication mean** $\bar{X}_i = \frac{1}{T} \sum_{j=1}^T X_j$. **Steady-state plot** = running avg vs #reps; flattens ⇒ enough reps (≈70 in Job Shop).

15 Required # replications – 3 methods

(a) **Rossetti half-width** (t-dist). Pilot $n_0=15$:

$$h_0 = t_{\alpha/2, n_0-1} \frac{s}{\sqrt{n_0}}, \quad n \approx n_0 \left(\frac{h_0}{h} \right)^2$$

unit of h = unit of KPI.
(b) **Ross iterative** – raise n_0 until

$$\Phi^{-1}(1 - \frac{\alpha}{2}) \frac{s}{\sqrt{n_0}} < h$$

(c) **Combined formula** (use when $n \geq 50$; pilot $n_0=5/10/15$):

$$n \geq \left(\frac{\Phi^{-1}(1 - \frac{\alpha}{2}) s}{h} \right)^2$$

Multiple KPIs: compute n per KPI, take *largest*; OR size on most important, report CIs for rest. **Smaller h ⇒ more reps.** **Practical:** start 20, inspect mean+half-width, usually 30/50/100 enough.

16 Worked: Rossetti half-width

Airport pilot: $n_0=15, \bar{x}=75.28, s=5.72, 95\% \text{ CI } (\alpha=0.05)$. With $t_{0.025,14} \approx 2.145$:
 $h_0 = 2.145 \cdot \frac{5.72}{\sqrt{15}} = 2.145 \cdot 1.477 \approx 3.17 \text{ min.}$
Desired $h=1.50 \text{ min} \Rightarrow n \approx 15 \left(\frac{3.17}{1.50} \right)^2 = 15 \cdot 4.46 \approx \mathbf{67 \text{ reps.}}$

17 Confidence interval per KPI

For $n \geq 30$ (std normal, $z=1.96$ at 95%):

$$\bar{x} \pm z \frac{s}{\sqrt{n}}, \quad z = \Phi^{-1}(1 - \frac{\alpha}{2})$$

small n : $\bar{x} \pm t_{\alpha/2, n-1} \frac{s}{\sqrt{n}}$. Half-width = ± term. Basis = **CLT**.

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

18 Worked: confidence interval

Job Shop FGI time, $n=98, \bar{x}=51.48, s=3.82, 95\% (z=1.96)$:
half-width = $\frac{1.96 \cdot 3.82}{\sqrt{98}} = \frac{7.49}{9.90} \approx 0.76$.
CI = $51.48 \pm 0.76 = [\mathbf{50.72, 52.24}] \text{ min.}$
Airport (67 reps): turnaround 72.61, $h=1.32 \Rightarrow [71.30, 73.93]$.

19 Time horizon

Terminating (finite): natural identical start/end each rep (bank 9–17). Method of **independent replications**; indep *across* reps.
Infinite (steady-state): no natural end, often *one long run*; within-rep data **not i.i.d.**; **warm-up period** (initial transient) must be discarded.

Terminating uses indep. replications
Steady-state needs warm-up removed
Within-rep data always i.i.d.

T
T
F

20 Comparing 2 configurations

Run n reps each (use **CRN**). Per rep:

$$D_j = X_{1j} - X_{2j}, \quad \bar{D} = \frac{1}{n} \sum D_j$$
$$S_D = \sqrt{\frac{1}{n-1} \sum (D_j - \bar{D})^2}, \quad \bar{D} \pm t_{\alpha/2, n-1} \frac{S_D}{\sqrt{n}}$$

3 outcomes: interval all $> 0 \Rightarrow$ config 1 better; all $< 0 \Rightarrow$ config 2 better; **contains 0** ⇒ no decision, need more reps.
If diff small but one config cheaper/simpler → choose it.

21 Worked: config comparison

$n=10$ paired reps, $\bar{D}=2.4 \text{ min}, S_D=3.1, 95\% (t_{0.025,9}=2.262)$:
 $\bar{D} \pm 2.262 \cdot \frac{3.1}{\sqrt{10}} = 2.4 \pm 2.262 \cdot 0.980 = 2.4 \pm 2.22$
= [0.18, 4.62]. All $> 0 \Rightarrow$ **config 1 statistically better.** (If it were [−0.3, 5.1] ⇒ contains 0 ⇒ no decision.)

22 Experimental design (Ph.3)

Organise as **base case** + alternative scenarios (scenario tree).

- **OFAT** (one factor at a time): vary one, fix rest. Simple; slow w/ many factors; **misses interactions**.
- **2^K factorial** ($K \geq 2$): 2 levels/factor = extremes; **detects interactions**; effects at several levels; assumes *linear* response over range; levels realistic but far apart.
- **Sim-based optimisation:** objective fn over outputs (profit/cost/wait) s.t. constraints (e.g. wait <5 min, util ≥80%); find best input combo.

Factor types: controllable (manager can set: #machines, reorder point) vs uncontrollable (arrival rate); quantitative (#resources – a number) vs qualitative (a routing rule / policy).

OFAT detects factor interactions
2^K factorial detects interactions
Factorial assumes linear response
#machines = quantitative factor

F
T
T
T

23 Worked: utilisation & throughput

Utilisation ρ = busy time / available time. A gate servicing aircraft 7.0 h out of a 12 h day: $\rho = 7/12 = \mathbf{58.3\%}$ (matches airport's 58.4%).
Throughput of serial line = min stage cap. If stages give 4000, 1200, 1100, 1000 bars/h ⇒ line outputs **1000/h** (the min).
Daily output = throughput × hours = $1000 \times 12 = 12,000 \text{ bars/day}$.

24 Distributions & means

- **Exponential** mean $1/\lambda$ – memoryless; inter-arrival times. $f(x) = \lambda e^{-\lambda x}$.
- **Poisson** = # events k in a period; $P(k) = \frac{\lambda^k e^{-\lambda}}{k!}$; $E[X] = Var[X] = \lambda$ (arrival rate).
- **Uniform**(a, b) mean $\frac{a+b}{2}$ – min/max only.
- **Triangular**(a, m, b) mean $\frac{a+m+b}{3}$ – expert estimate.
- **Normal** symmetric; **Erlang** = sum of exponentials.

Link: Poisson arrivals (rate λ) ⇔ exponential inter-arrival times (mean $1/\lambda$).

25 TUTORIAL 1 – Bank Office

Customers arrive, choose **ATM** or one of **4 tellers**; FIFO queue each. AnyLogic shows *how* not *why* ⇒ reverse-engineer Phase 1.

- **Entity** = customer; **resources** = tellers + ATM.
- **Goal:** balance service level vs operational cost (don't over-pay for resources).
- **Two bottlenecks:** #tellers + the single ATM. Cut 4→3 tellers ⇒ waiting time up. Rising ATM demand ⇒ add 2nd ATM.
- **KPIs:** ATM utilisation (%), avg queue length, waiting-time distribution, time in system; + teller utilisation.
- **Lesson:** *no single correct model* – framing matters more than numbers.

26 TUTORIAL 2 – YummiBar line

dough→mixer→moulder→cooler→pack(wrap, box of 24, stack of 20). Owns 1 mixer, 2 moulders, 3 coolers, old + new pack line. **Caps (bars/h):** Mixer 4000, Moulder 1200(×2), Cooler 1100(×3), Old pack 1000, New pack 2000. Dough 150 kg/h, 50 g/bar ⇒ 3000 bars/h in; 12 h/day.

Entity choice: tray of **20 bars** (= box). 480-pallet rejected: machine caps not divisible by 480, units processed whole. 2nd agent "pallet" = 24 trays = 480 bars for final stage.
Delay per machine = $\frac{1}{\text{cap}/20}$ (one unit = 20 bars, 1 at a time). **Time unit** = minutes (sec too fine, hr too coarse). Build *one machine at a time*.

Bottleneck: serial throughput = **min** stage. New branch cap ≈1200 (moulder); old branch cap **1000** (old packaging = bottleneck). **Upgrade Moulder 1** (binding stage on busy branch); non-bottleneck upgrade = useless.

27 TUTORIAL 3 – Airport ground handling

Aircraft (entities) land → allocated to **gates** (resources) → ground handling → depart. System: enter airport/transfer queue → leave gate. **Gates = suspected bottleneck** (track queue time + utilisation). Built over 3 days: (1) gate alloc + refuel, (2) baggage (after refuel, needs baggage truck) + varying fuel trucks, (3) input modelling + output analysis.

- **Refuel delay** = (tank – residual fuel)/speed; tank = 200,000 if pax≥150 else 150,000.
- **Gate** = static ResourcePool; **fuel truck** = dynamic pool (e.g. 2). Aircraft seizes a truck.
- **Input modelling:** fit dist to RefuellingData (time/litre) + BaggageHandlingData (time/bag) via Fitter.
- **Downtime block** (stuck bag): inter-arrival 30 min, dur $\sim N(90s, \sigma=\mu/10)$.
- **Datasets:** TestSchedule (small, build/test) vs FullSchedule (peak traffic, real runs).
- **Assumption trap:** wait between 2 ops counted as operation time ⇒ queue time understated if all trucks busy. Fix both diagram + assumptions.

Result (67 reps, 95%): turnaround 72.61±1.32, queue 21.03±0.64, util 58.4%±0.7% – steady state.

All 3: DEGREE + 6 phases; Phase 1 (problem, system, KPIs, conceptual model, assumptions) *before* coding. Entity = what flows; resource = what constrains; bottleneck = slowest stage = only worthwhile upgrade. KPI defs & assumptions shape results.

28 Master T/F + exam traps

Simulation gives only estimates
DES = stochastic+dynamic+discrete
Verification = model vs specification
Validation = model vs reality
Bottleneck caps system throughput
 χ^2 /KS can *rank* fits
Empirical dist limited to data range
Smaller half-width ⇒ more reps
CI rests on Central Limit Theorem
Steady-state needs warm-up removal
Comparison interval $\ni 0 \Rightarrow$ decide
OFAT misses interactions
2^K factorial detects interactions
CRN makes comparison fairer

T
T
T
T
T
F
T
T
T
F
T
T

Top traps: (1) **Verify ≠ validate** = spec vs reality. (2) h_0 **experienced (pilot) ≠ h desired** in $n \approx n_0(h_0/h)^2$. (3) GoF tests **reject** a fit, can't **rank**. (4) Comparison interval $\ni 0 \Rightarrow$ **no decision**. (5) Empirical dist can't exceed observed range. (6) Upgrade **only** the bottleneck. (7) Static = Monte Carlo; DES = events. (8) V&V loop in Ph.2; "more runs?" Ph.4.

29 V&V techniques (how, in practice)

Verification checks: #agents in = #agents out; agents created with correct params from arrival scheme; operations entered in right order; resources seized & released correctly; observed delay times match the dist mean. *Tools:* print to console, animation, **stress-test** (sensitivity analysis: raise demand / cut resources).
Validation = **3-step approach** (conceptually distinct, done together): (1) build a model with high **face validity**; (2) validate the **assumptions**; (3) compare the model's **input–output transformation** to the real system's. Show to personnel for sign-off; compare output to real data.

30 KPI definitions (quick reference)

- **Utilisation** = busy time / available time (% a resource is in use).
- **Throughput** = units completed per time; serial line = min stage cap.
- **Queue/waiting time** = time an entity waits for a resource.
- **Lead/turnaround time** = total time in system (entry→exit).
- **Availability** = available equip / total; **on-time rate** = on-time / total.